ATS Resume Matcher: NLP & ML Project Overview

College-Level Introduction to Machine Learning (IML) Project | Team Preparation Guide

1. Executive Summary & Objective

Automated Applicant Tracking Systems (ATS) screen resumes against job descriptions to identify qualified candidates. This project implements a 100% local, transparent, and explainable ATS matching engine using foundational Natural Language Processing (NLP) and an unsupervised Vector Space Model (VSM).

Key Academic Positioning: Rather than fabricating a fictitious 99% accurate neural network trained on nonexistent labeled hiring datasets, this project formulates resume screening correctly as an **unsupervised document similarity and information retrieval task**. It fits TF-IDF directly on compared document vocabularies, measures directional cosine similarity, extracts domain skills via punctuation-safe regular expressions, and highlights the exact keywords that influenced the score.

2. Technical Stack & Architecture

Component	Technology	Purpose / Justification
Frontend UI	Streamlit (Light Theme)	Interactive dashboard with light mode, high contrast, zero glassmorphism.
NLP Vectorization	scikit-learn TfidfVectorizer	Unigrams + Bigrams (1,2), sublinear log scaling, smooth IDF.
Similarity Metric	scikit-learn Cosine Similarity	Length-invariant vector dot product on L2-normalized vectors.
PDF Extraction	PyMuPDF (fitz)	High-speed C-level digital text extraction handling corrupt/empty files.
Data & Visuals	Pandas & Plotly	Tabular benchmarking, gauge charts, and candidate comparison bars.
Test Framework	pytest (53 tests)	Comprehensive unit and integration test suite passing at 100%.

3. What the Website Does (Core Features)

- **PDF Resume Upload & Text Extraction:** Direct drag-and-drop PDF extraction with character/word counting and corrupt/scanned PDF warnings.
- **Preloaded Benchmark Roles:** Instant selection of 5 industry job descriptions (Data Scientist, ML Engineer, Full Stack, Data Analyst, Software Engineer).
- **Preloaded Candidate Profiles:** Instant loading of 5 diverse fictional candidate profiles (Alice, Bob, Carol, David, Eva) for immediate testing.
- **Punctuation-Aware Skill Extraction:** Extracts hard skills into **Matched Skills** (green), **Missing Skills** (red), and **Additional Skills** (blue).
- **Configurable Score Weighting:** Interactive sliders to adjust the relative weight of TF-IDF Cosine Similarity vs. Explicit Skill Coverage.
- **Explainable AI (XAI) Panel:** Renders the top overlapping n-grams and their exact vector dot products (u_i * v_i) explaining the score.
- **Actionable Gap Recommendations:** Automatically generates tailored advice based on missing skills and resume text density.

• **Multi-Candidate Benchmark Ranking:** Compares all 5 fictional resumes simultaneously against any role with Plotly horizontal bar charts.

• **Model Transparency & Viva Tab:** Complete in-app interactive study guide with 16 viva questions and LaTeX mathematical formulations.

4. How It Does It: Step-by-Step NLP Pipeline

Step 1: Text Extraction & Normalization: The user's PDF is read into an in-memory byte stream. PyMuPDF inspects each page's digital text layer. Control characters and null bytes are stripped, and whitespace is normalized.

Step 2: Punctuation-Aware Skill Matching: The text is evaluated against an 8-category skill taxonomy using negative lookaround regexes (?<![a-zA-Z0-9+#])c(?![a-zA-Z0-9+#]). This ensures C++ or C# never trigger C falsely. Matched, missing, and additional skills are partitioned using Python set operations.

Step 3: Sublinear TF-IDF Vectorization: Both documents are transformed into sparse vectors using scikit-learn's TfidfVectorizer with ngram_range=(1, 2) and English stop-word filtering. Sublinear scaling calculates tf = 1 + ln(tf), dampening keyword stuffing.

Step 4: Cosine Similarity Computation: The L2-normalized vectors are multiplied via Euclidean dot product: cos(u, v) = sum(u_i * v_i). This yields a length-invariant directional similarity score between 0.0 and 1.0.

Step 5: Composite Scoring & Explainability: The final ATS score is computed as: ATS Score = w_tfidf * CosineSim + w_skill * (MatchedSkills / JDSkills). Top shared terms are extracted from non-zero products u_i * v_i and displayed in descending order.

5. Mathematical Formulations (Key Formulas to Memorize)

Concept	Formula	Explanation
Sublinear TF	`tf_scaled = 1 + ln(tf)`	Applies logarithmic smoothing to raw word counts, suppressing keyword-stuffing exploits.
Smooth IDF	`idf(t) = ln((1+n)/(1+df(t))) + 1`	Down-weights common words that appear across all documents while rewarding distinctive domain skills.
Cosine Similarity	`cos(u, v) = (u . v) / (||u|| * ||v||)`	Evaluates the angle between normalized document vectors, making scoring independent of resume length.
Skill Coverage	`Coverage = |S_res ∩ S_jd| / |S_jd|`	Measures the exact proportion of hard technical requirements satisfied by the candidate.
Composite ATS	`Score = w_t * CosSim + w_s * Coverage`	Blends general contextual alignment with non-negotiable technical skill compliance.

6. Candidate Benchmark Pool (Model Discrimination)

Candidate	Profile Summary	Expected Behavior
Eva Chen	AI/ML Engineer: Transformers, LLMs, PyTorch, MLOps, AWS	Ranks #1 on ML Engineer and Data Science roles (~75-80% coverage).
Carol Smith	M.Sc. Data Science: Spark, PyTorch, SQL, Statistics, A/B Testing	Strong fit for Data Science and Senior Analyst roles (~60-65% coverage).
Alice Johnson	College Grad: Python, scikit-learn, EDA, Pandas, basic NLP	Solid junior match for Data Science roles (~50-55% coverage).
Bob Martinez	Full Stack Dev: React, Node.js, Express, Docker, Django	Dominates Full Stack role; accurately scores low on Data Science.
David Lee	IT Support Fresher: Basic C++, Java, Excel, IT support	Accurately receives lowest scores (<15%), demonstrating model selectivity.

7. Top 10 High-Priority Viva Questions & Answers

Teammates should study these 10 core questions carefully for project viva examinations:

Q1: Why is this project unsupervised rather than using a trained supervised classifier?

Model Answer: Supervised hiring models require thousands of labeled (Resume, Job Description, Hired/Rejected) pairs, which are unavailable due to privacy laws and suffer from historical hiring bias. An unsupervised Vector Space Model directly evaluates document similarity without needing synthetic labels, generalizing dynamically to any job description.

Q2: Why use Cosine Similarity instead of Euclidean Distance?

Model Answer: Euclidean distance is biased by document length (a 3-page resume would be distant from a 1-page resume even if identical in topic). Cosine similarity measures the angle between normalized vectors, making it completely length-invariant.

Q3: What does sublinear TF scaling do?

Model Answer: It computes tf_scaled = 1 + ln(tf). This prevents candidates from exploiting keyword stuffing by repeating a word 20 times to artificially inflate their similarity score.

Q4: Why are bigrams (1, 2) included alongside unigrams?

Model Answer: Unigrams like 'machine' and 'learning' lose their meaning when separated. Bigrams capture crucial compound phrases like 'machine learning', 'data science', and 'random forest'.

Q5: How does the system handle punctuation in skills like C++, C#, and .NET?

Model Answer: Standard tokenizers strip punctuation, reducing C++ and C# to 'c'. Our engine uses negative lookaround regexes '(?<![a-zA-Z0-9+#])c(?![a-zA-Z0-9+#])', guaranteeing 'C' will never match inside 'C++' or 'C#'.

Q6: Why combine TF-IDF Cosine Similarity and Skill Coverage?

Model Answer: TF-IDF captures overall vocabulary and descriptive congruence, while Skill Coverage checks for mandatory non-negotiable technical skills. A weighted combination gives recruiters a balanced picture of both fit and qualifications.

Q7: How does Explainable AI (XAI) work in this dashboard?

Model Answer: Because cosine similarity on normalized vectors equals sum(u_i * v_i), each term's contribution is simply its dot product u_i * v_i. We extract and sort the highest product terms to show the exact keywords that generated the score.

Q8: What are the limitations of pure TF-IDF matching?

Model Answer: TF-IDF is lexical; it cannot detect synonyms (e.g., 'developer' vs 'software engineer') or polysemy ('Apple' fruit vs 'Apple' company). Dense embeddings like Sentence-BERT can resolve this in future enhancements.

Q9: How are corrupt, encrypted, or scanned PDFs handled?

Model Answer: PyMuPDF catches stream corruption errors, checks doc.needs_pass for password protection, and flags image-only scanned PDFs that lack a text layer.

Q10: What is the computational complexity of the matching pipeline?

Model Answer: Linear time O(L) where L is the total word count of the documents. Vectorization and cosine similarity execute in under 25 milliseconds on CPU.

8. Quick Windows Execution Commands

Activate virtual environment & launch dashboard:

```
cd C:\MatchMyResume
.\venv\Scripts\activate
python -m streamlit run app.py
```

Run automated test suite:

```
python -m pytest tests/ -v
```